Fully Probabilistic Deep Models for High-Dimensional Nonlinear PDEs: Challenges and Insights

Jerry Zhen

Abstract: Numerical simulation of differential systems has long been a challenge, and recent advances have explored the use of machine learning techniques—particularly neural networks—to address these issues. However, despite notable progress, many existing approaches remain largely data-driven and often limited to relatively simple problems. To better understand neural partial differential equation (PDE) solvers for complex systems, we implemented a physics-driven deep latent variable model (PDDLVM) [10] and tested it on the phase-field crystal (PFC) [1] model, which is a sixth-order nonlinear PDE. Although the model failed to converge, we gained valuable insights and highlighted several challenges in training and fitting such models. To provide a comparative perspective, we also evaluated the performance of traditional Physics-Informed Neural Networks (PINNs)[7] and the Fourier Neural Operator (FNO)[5] on the same task and have found intriguing directions to explore.

1. Introduction

Partial differential equations (PDEs) are foundational tools for modelling physical behaviours, ranging from fluid dynamics to chemical reactions. However, they are notoriously difficult to solve, and most problems can only be approximated using numerical methods such as finite element methods (FEM). To address these issues, researchers have tried to approach them, and machine learning techniques, especially deep neural networks, have emerged as a potent approach. Specifically, in the paper "Physics-informed neural networks (PINNs)," the author proposed a novel framework that approximates the solutions of PDEs using neural networks by embedding physical losses as a soft constraint to minimize. Consequently, this paper inspired many works and has significantly advanced the development of neural solvers for PDEs. However, despite the impressive progress, many illustrated examples are relatively simple and rely on simulated or observed data. In addition, there is often little discussion about the scalability of the proposed method for high-dimensional problems and its performance for complex PDEs, where neural networks could truly excel. To examine these challenges, we implemented a neural PDE solver based on the PDDLVM framework—an unsupervised training strategy that enables uncertainty quantification in an attempt to solve the phase-field crystal (PFC) model, a sixth-order nonlinear partial differential equation. We highlight common problems in its implementations that are usually overlooked in simplified studies, as well as our attempt to resolve them. We further provide a discussion on possible directions for improvements, especially how Fourier Neural Operator (FNO) and other neural operators could be worth exploring to integrate into the PDDLVM framework. The remaining work is structured as follows: Section 2: an introduction to PDEs, PINN, PDDLVM, PFC model, and FNO. In Section 3, we delve into

the specifics of the PINN-PDDLVM implementation and discuss the difficulties we faced. In Section 4, we present the experimental results. Section 5 provides a summary of the experiment and suggests potential directions for further exploration.

2. Preliminaries

In this section, we briefly review the definition of partial differential equations and introduce the concept of PINN, PDDLVM, the building blocks of our neural PDE solvers. We then state the PFC model, the differential equations we want to solve, and the Fourier Neural Operator, a neural network architecture.

2.1. Partial Differential Equations (PDEs)

In this work, we focus on spatiotemporal partial differential equations. A partial differential equation (PDE) is a mathematical equation that involves two or more independent variables, an unknown function u, and the partial derivatives of u with respect to the independent variables. Formally, we adapt the notation from [12] and write

$$u_t + \mathscr{N}[u] = 0, \quad t \in [0,T],\ x \in \Omega \subset \mathbb{R}^d \tag{1}$$

subject to initial and boundary conditions

$$u(0,x) = g(x), \quad x \in \Omega \tag{2}$$

$$\mathscr{B}[u](t,x) = 0, \quad t \in [0,T],\ x \in \partial\Omega \tag{3}$$

where $\mathscr{N}$ is some differential operator, $\mathscr{B}$ a boundary operator, $g(x)$ a known function for the initial condition, and T the stopping time. We further define residual functions. $R[u]$ as

$$R[u](t,x) = u_t(t,x) + \mathscr{N}[u](t,x) \tag{4}$$

which evaluates to 0 for u that solves the differential system.

2.2. Physics-Informed Neural Networks (PINNs)

PINN is a framework to approximate the solutions of a PDE system using neural networks. Denote the approximation by $\hat{u}$. We have

$$\hat{u}(x) = (\ell_n \circ \cdots \ell_i \circ \cdots \ell_0)(x), \quad \ell_i = \sigma(\mathrm{W}_i x + b_i) \tag{5}$$

here, W_i is the weight matrix, b_i is the bias vector, and σ denotes the activation function, such as $\tanh(x)$. Now, define the initial, boundary, and residual loss by

$$\mathscr{L}_{ic}(\hat{u}) = \frac{1}{N_{ic}} \sum_{i=1}^{N_{ic}} \left| \hat{u}(0, x_{ic}^i) - g(x_{ic}^i) \right|^2 \tag{6}$$

$$\mathscr{L}_{bc}(\hat{u}) = \frac{1}{N_{bc}} \sum_{i=1}^{N_{bc}} \left| \mathscr{B}[\hat{u}](t_{bc}^i, x_{bc}^i) \right|^2 \tag{7}$$

$$\mathscr{L}_r(\hat{u}) = \frac{1}{N_r} \sum_{i=1}^{N_r} \left| R[\hat{u}](t_r^i, x_r^i) \right|^2. \tag{8}$$

where $\{x_{ic}^i\}_{i=1}^{N_{ic}}, \{t_{bc}, x_{bc}^i\}_{i=1}^{N_{bc}}, \{t_r^i, x_r^i\}_{i=1}^{N_r}$ are the initial, boundary, and residual spatiotemporal coordinates. We want to tune W_i, b_i's to obtain optimal $\hat{u}$ that minimizes the loss function:

$$\mathscr{L}(\hat{u}) = \mathscr{L}_{ic}(\hat{u}) + \mathscr{L}_{bc}(\hat{u}) + \mathscr{L}_r(\hat{u}) \tag{9}$$

2.3. *Physics-Driven Deep Latent Variable Model (PDDLVM)*

In this section, we introduce a simplified[1] PDDLVM framework that aims to establish a probabilistic mapping from spatiotemporal coordinates to a conditional distribution of solutions. In essence, this framework assumes u, the solution of PDE, to be a random variable conditioned on inputs (t,x) such that

$$u|t,x \sim \mathcal{N}\left(\mu_\alpha(t,x), \sigma_\alpha^2(t,x)\right) \tag{10}$$

where $\mu_\alpha(t,x), \sigma_\alpha(t,x)$ are estimated using α-net, a neural network:

$$(t,x) \overset{\alpha\text{-net}}{\longmapsto} \left(\mu_\alpha(t,x), \sigma_\alpha(t,x)\right) \tag{11}$$

and assumes $u|t,x$ are independent at different spatiotemporal points $(t,x) \in [0,T] \times \Omega$. Note in this setup, one can easily perform uncertainty quantification. To perform inference, we simply approximate u by the most probable prediction $\mu_\alpha(t,x)$ or perform a random draw.

For training, we train α-net to maximize the following objective function:

$$\mathbb{E}_{q_\alpha}[\log p(r=0|u) - \log q_\alpha(u|t,x)] \tag{12}$$

where

$$r|u \sim \mathcal{N}(R[u], \sigma_r) \tag{13}$$

and $q_\alpha(u|t,x)$ is the conditional distribution proposed in 10. To compute 12, we simply make draws from $q_{\alpha(u|t,x)}$ and use Monte-Carlo approximation to obtain the expectation. A detailed implementation will be discussed in section 3.

Intuitively, the $\log p(r=0|u)$ term serves as a physical constraint, as it's maximized when $R[u]$ equals 0. On the other hand, the $\log q_\alpha(u|t,x)$ serves as a regularization term that prevents overconfidence by encouraging a larger σ_α (so that $q_\alpha(u|t,x)$ is small and 12 becomes large). Combining the two losses, we are effectively forcing improvements over the objective function to come from a more accurate $\mu_\alpha(t,x)$ instead of a smaller $\sigma_\alpha(t,x)$.

2.4. *Phase-Field Crystal (PFC) Model*

The Phase-Field Crystal (PFC) model is a framework for modeling crystalline materials that can be used to model the reaction process of molten-salt reactor. Its mathematical formulation consists of:

- **Free energy functional**:

$$\mathrm{F}[\psi] = \int \left[\frac{1}{2}\psi\left((1+\nabla^2)^2 - \gamma\right)\psi + \frac{\psi^4}{4}\right] d\mathbf{x} \tag{14}$$

- **Dynamical equation**:

$$\frac{\partial \psi}{\partial t} = \nabla^2 \frac{\delta \mathrm{F}}{\delta \psi} = \nabla^2([(1+\nabla^2)^2 - \gamma]\psi + \psi^3) \tag{15}$$

The dynamic equation is the differential system we aim to solve where we assume a periodic boundary condition. When fully expanded, this equation contains a nonlinear cubic term ψ^3 and sixth-order spatial derivatives $\nabla^6\psi$, making it extremely challenging to solve. In practice, it is solved using semi-implicit Fourier-spectral method.

[1] The full PDDLVM framework handles both forward/inverse mappings with probabilistic uncertainty quantification. This simplified version considers only the forward mapping, as we wanted to observe model convergence (which had failed) before adding complexity. The complete dual-network architecture with variational inference is detailed in [10]

2.5. *Fourier Neural Operator (FNO) for PDEs*

The Fourier Neural Operator (FNO) is a neural architecture designed to learn solution operators for PDEs. Given a PDE state $u_{curr}(x)$, discritized as a 2D grid, the FNO predicts the next state $u_{next}(x)$ auto-regressively. Its structure comprises:

1. Encoder: encodes u_{curr} into a high-dimension space $v_0(x)$ by multiple layer perceptron (MLP)

2. Fourier Layers: A series of transformations performed in the Fourier domain. Each layer updates $v_l \to v_{l+1}$ via

$$v_{l+1}(x) = \sigma\left(W_l v_l(x) + \mathscr{F}^{-1}(R_l \cdot \mathscr{F}(v_l))(x)\right)$$

   where $\mathscr{F}, \mathscr{F}^{-1}$ denote Fourier/inverse Fourier transforms, R_l, W_l learnable linear transformations for $\mathscr{F}(v_l)$ and $v_l(x)$ and σ the activation function. Note we discard high-frequency modes in $F(v_l)$ to avoid aliasing and improve stability.

3. Decoder: decodes v_f the output of the last Fourier layer into the next step prediction u_{next} by MLP.

3. Methodology

In this section, we cover the implementation of PDDLVM and some challenges encountered that are rarely discussed in 1-dimensional or relatively simple PDEs.

3.1. *Model Structure*

We implemented a **multi-headed neural network** that maps inputs (t,x) to the parameters of a Gaussian distribution as described in 2.3. For a multi-headed neural network, we have a shared base of the network consist of k fully connected layers denoted by $\ell_1, \ldots, \ell_k$ where each $\ell_i = \phi_i(W_i x + b_i)$. Define

$$x_{\text{processed}} = (\ell_n \circ \ell_{n-1} \circ \cdots \circ \ell_1)(x) \tag{16}$$

we compute

$$\mu = \ell_\mu(x_{\text{processed}}), \quad \log(\sigma) = \ell_\sigma(x_{\text{processed}}) \tag{17}$$

where ℓ_μ, ℓ_σ are simply linear transformations without an activation.

3.2. *Data Preparation*

In the original work, the proposed methodology was successful for 1D spatiotemporal systems (e.g., the 1D heat equation) by training on all collocation points across the full spatiotemporal domain simultaneously. However, this approach becomes computationally infeasible for higher-dimensional problems due to the **curse of dimensionality**[2]. To address this issue, we:

- Partition the time domain into uniform intervals: $(t_0, t_1], \ldots (t_{k-1}, t_k]$, $t_0 = 0$ and $t_k = T$.
- Sample $t^i \in (t_{i-1}, t_i]$ for $i = 1, \cdots k$, denote $\tau = \{t^1, \ldots, t^k\}$
- Denote $\Omega \subset \mathbb{R}^2$ the set of all discritized spatial coordinates, we train the neural network on the data set $\{(t^i, x) | x \in \Omega, i = 1, \ldots, k\}$. This is effectively the set of all spatial coordinates at the selected time frames.

[2]The spatiotemporal resolution must be reduced drastically to maintain the number of collocation points as dimension increases.

Note that [2] proposed a strategy to deal with high-dimension PDEs. Nonetheless, it still relies on computing all the partial derivatives, which does not extend to high-order differential equations.

3.3. Residual Computation

Recall the residual is defined as

$$R[u](t,x) = u_t(t,x) + \mathscr{N}[u](t,x) \tag{18}$$

where the differential operator $\mathscr{N}$ typically requires spatial differentiation. While automatic differentiation suffices for low-order derivatives, it becomes prohibitive for sixth-order derivatives in the PFC model (Section 2.4) due to the extraordinary amount of memory consumption and computation time. To resolve the issue, we exploit Fourier spectral differentiation:

$$\begin{aligned}
\mathscr{N}[\psi] &= \nabla^2\left([(1+\nabla^2)^2-\gamma]\psi+\psi^3\right) && (19)\\
&= \mathscr{F}^{-1}\left(\mathscr{F}\left(\nabla^2\left([(1+\nabla^2)^2-\gamma]\psi+\psi^3\right)\right)\right) && (20)\\
&= \mathscr{F}^{-1}\left(-k^2\left[\underbrace{(1-k^2)^2-\gamma}_{\text{Linear term}}\mathscr{F}(\psi)+\underbrace{\mathscr{F}(\psi^3)_{\text{dealias}}}_{\text{Nonlinear term}}\right]\right) && (21)
\end{aligned}$$

Where:

- $\mathscr{F}$, $\mathscr{F}^{-1}$ are Fourier/inverse Fourier transforms
- k is the wavenumber vector in Fourier space ($k = \|\mathbf{k}\|$ for isotropic systems)
- The dealiased nonlinear term $\mathscr{F}(\psi^3)_{\text{dealias}}$ uses 2/3 truncation to prevent aliasing errors

After obtaining $\mathscr{N}[\psi]$, we compute u_t using automatic differentiation and consequently the residual $R[u](t,x)$. Finally, for this method to work, we need to have all spatial points at a particular frame, and this is satisfied by the data preparation process.

3.4. Loss Functions

Recall that

- $x \in \Omega \subset \mathbb{R}^2$ and $\tau = \{t^1, \ldots, t^k\}$ the selected time.
- $\mu_\alpha, \sigma_\alpha$ the predicted mean and variance from α-net.
- $N(\cdot;\mu,\varepsilon^2)$ the normal distribution cantered at μ with variance ε^2
- $\psi_{ic}(x)$ the initial condition at x
- $\mathscr{B}$ the boundary operator 3 and R the residual function 4

we define the following loss functions:

- **Initial Condition Loss**:

$$\mathscr{L}_{\text{ic}} = -\sum_{x\in\Omega} \log N(0;\mu_\alpha(0,x)-\psi_{ic}(x),\sigma_{ic}^2) \quad \sigma_{ic} = 0.001 \tag{22}$$

- **Boundary Condition Loss**:

$$\mathscr{L}_{\text{bc}} = -\sum_{\substack{t\in\tau \\ x\in\partial\Omega}} \log N\big(0; \mathscr{B}[u](t,x), \sigma_{bc}^2\big), \quad \sigma_{bc} = 0.001 \tag{23}$$

- **Residual Loss**:

$$\mathscr{L}_{\text{r}} = \sum_{\substack{t\in\tau \\ x\in\partial\Omega}} \log N\big(\hat{u}(t,x); \mu_\alpha(t,x), \sigma_\alpha^2(t,x)\big) - \sum_{\substack{t\in\tau \\ x\in\partial\Omega}} \log N\big(0; R[u](t,x), \sigma_r\big) \tag{24}$$

$$\hat{u}(t,x) = \mu_\alpha(t,x) + \sigma_\alpha(t,x)\varepsilon, \quad \varepsilon \sim N(0,1)), \quad \sigma_r = 0.01 \tag{25}$$

and minimize the weighted sum of the three losses

3.5. pseudo code

To summarize, we train α-net according to the following procedure

Algorithm 1 PDDLVM Training Procedure

Require: Initial α-net, number of iterations num_step
Ensure: Trained α-net

```
Initialize α-net
for step ← 1 to num_step do
    Sample t^i from (t_{i-1}, t_i) for i = 1, ..., k
    Compute μ_α(t^i, x), σ_α(t^i, x) ∀x ∈ Ω, i = 1, ..., k
    Compute loss terms:
        L^i_ic (initial condition loss)
        L^i_bc (boundary condition loss)
        L^i_r (residual loss)
    Compute total loss L_total = weighted average(Σ_i L^i_ic + Σ_i L^i_bc + Σ_i L^i_r)
    Update α-net parameters using L_total
end for
```

4. Results

In this section, we summarize the experimental results and observations.

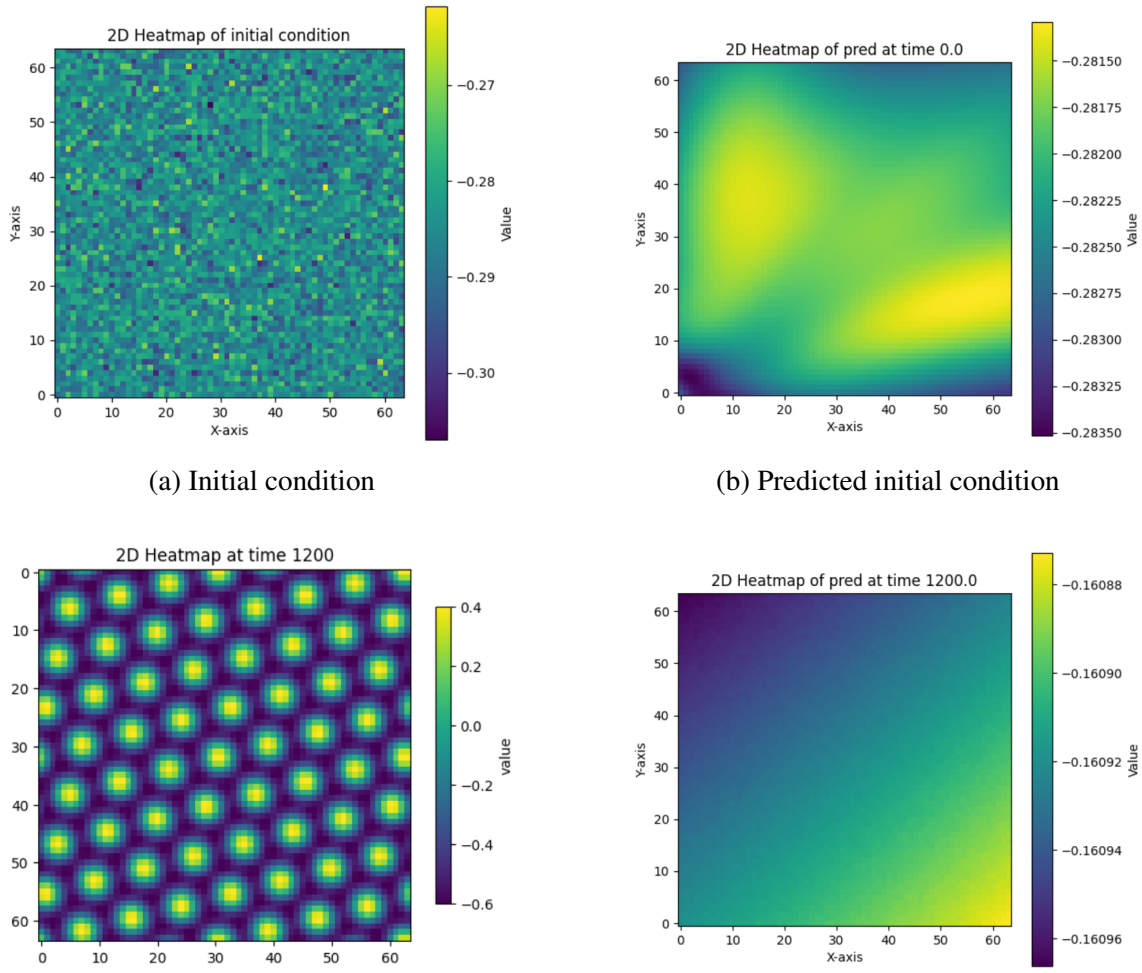
4.1. Set up

We implemented all the neural networks under PyTorch 12.4 and conducted all the training using a 12700k and a 3070 GPU (8 GB RAM). Due to limited RAM available, we sample around 200 time steps for each batch and use a 4 shared linear layer with $[128, 64, 64, 64]$ nodes and 2 separate linear layers $[64, 1]$ that compute respectively μ_α and σ_α. We used the tanh() activation function. The initial condition used is

$$-0.285 + (0.285 \cdot 0.02) \cdot \varepsilon, \quad \varepsilon \sim N(0,1) \tag{26}$$

4.2. *PDDLVM on PFC model*

As stated earlier, the PDDLVM model failed[3] to converge for the PFC model. In particular, we observed a massive residual loss in the beginning of the training. It's usually above the order of 10^{13}, whereas the initial loss is around the order of 10^4. Given the interpretation of our loss, the negative evidence lower bound on the log evidence $r = 0$, it's not surprising to see extremely large loss in the beginning, as that simply implies $p(r = 0) = 0$. Boundary loss, on the other hand, quickly converged to near 0, but that's only because the neural network degenerated into predicting constants at each time frame.



(a) Initial condition

(b) Predicted initial condition

(c) Ground truth at $t = 1200$

(d) Prediction at $t = 1200$

Fig. 1. Comparison between ground truth and predictions using a trained PDDLVM.

Figure [1] showed the ground truth at $t = 1200$ exhibits a periodic pattern, whereas the prediction is predicting almost a constant (for each time frame). Intuitively, this is because we have a very dominant residual loss. It's so dominant such that for any "direction" the parameter moves, it will be moving in the direction of a degenerate solution, which can greatly reduce the loss in the first epoch and trap the consequent convergence to be at best locally optimal. In fact, we observed the initial condition loss to not decrease even when the residual loss got smaller than the initial condition loss, suggesting local maxima. Alternatively, the neural network might

[3]Despite the failure of the PFC model, we successfully trained the α-net for the 1-D heat equation problem

just be too simple to capture the complex dynamic of such high-order nonlinear differential systems.

4.3. PINN on PFC model

Given the failure of PDDLVM, we turned to PINN and want to examine if it can converge in this simpler framework. However, we observed a similar trend as in the PDDLVM framework. In fact, the loss function of PDDLVM can be seen as a transformation of PINN, as the residuals are computed in a similar manner. The only difference lies in that we directly compute mean squared loss on the residual for PINN and compute the average of log norm probability for PDDLVM, which produces. However, with the assumption that $r = 0$ and $r|u \sim N(R[u], \sigma_r^2)$, one term in the the residual loss becomes

$$\mathscr{L}_r = \sum \left(\ln \left(\frac{1}{\sigma_r \sqrt{2\pi}} \right) - \frac{1}{2} \left(\frac{0 - R[\hat{u}]}{\sigma_r} \right)^2 \right) \tag{27}$$

which is the weighted mean-squared loss plus some constants. Thus, we are not surprised if PINN too failed to converge. We further tested if including a data loss can help the model to converge correctly. In particular, we define

$$L_{obs}(\hat{u}) = \frac{1}{N_{obs}} \sum_{i=1}^{N_{obs}} \left| R[\hat{u}](t_{obs}^i, x_{obs}^i) \right|^2 \tag{28}$$

and add that to the total loss. Nevertheless, this attempt failed, too.

4.4. Fourier Neural Operator on PFC model

Unlike the other two neural networks that approximate u. FNO predicts the next PDE state from the current state. We trained FNO on some simulated data with time step $\Delta t = 0.1$ and found that we actually achieved some results; that is, we achieved a relatively small error for a single-step prediction.

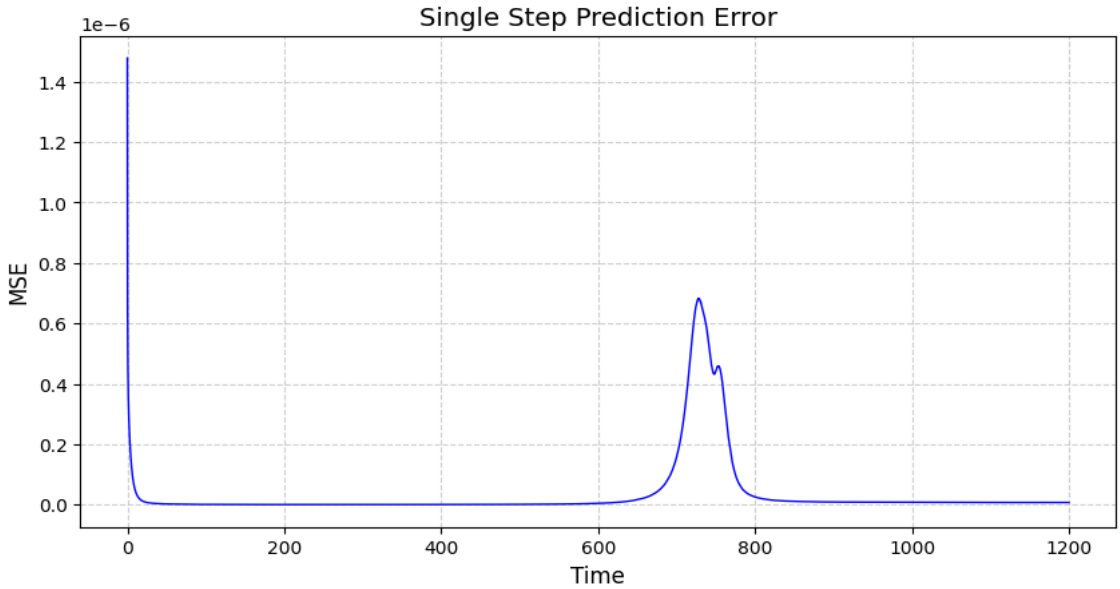


Fig. 2. The single Step Prediction error of FNO

By observation, the y-axis of figure [2] is on the magnitude of 10^{-6}, suggesting a fairly small single-step prediction. Interestingly, we can observe a bump at around $t = 750$. This spike is likely caused by a sudden free energy drop around that time period, as shown in [3]. When the free energy drops dramatically, the density field must be undergoing some drastic change (i.e., a very sharp gradient at time $t \approx 750$). Hence, it's reasonable that the FNO model failed to capture this abrupt change. Although it's impressive to learn the neural operator might be capturing some short-term behaviours of the model, it immediately failed when "extending" this behaviour to long-term predictions where it does inference in an autoregressive manner. In figure [4], we plotted the MSE between the autoregressive predictions and the expectations. We observe the error begin to explode right after $t = 20$ seconds.

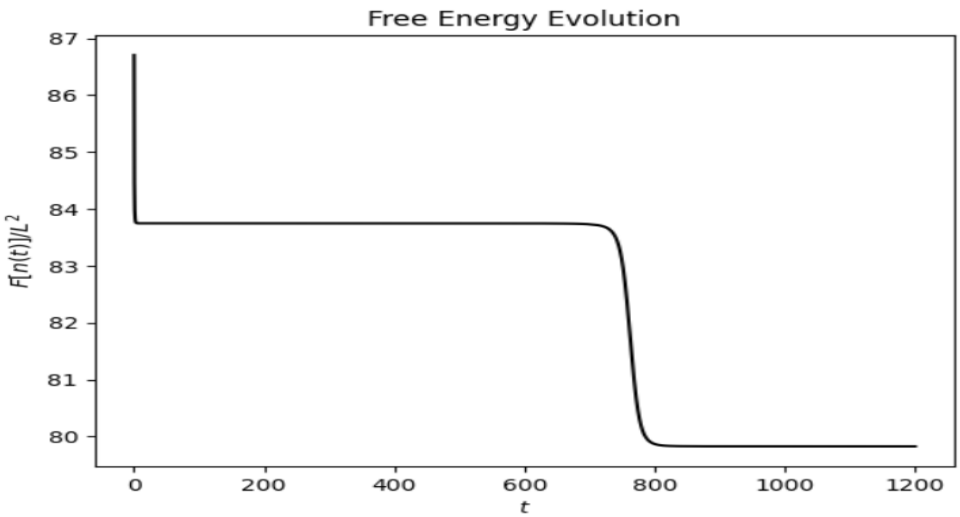


Fig. 3. The Free Energy Evolution of the PDE system

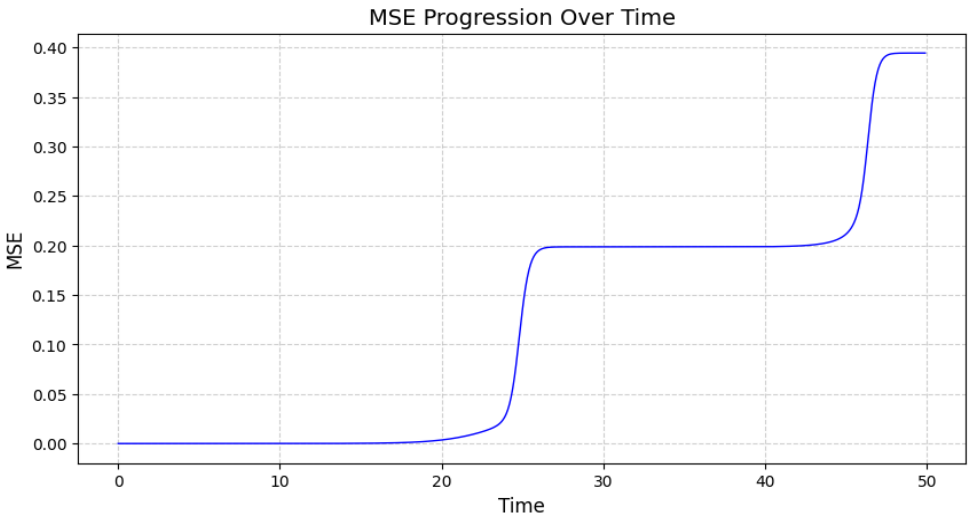


Fig. 4. The error in performing auto-regressive prediction

5. Conclusion and discussion

In this work, we have examined the use of advanced Bayesian concepts in a novel setting. In particular, how one can construct a loss function for the neural PDE solver using variational approximation and Monte Carlo approximation. Despite the model failing to converge, it still provided us plenty of valuable insights into caveats to train neural networks to solve high-order nonlinear PDEs. In particular, we witnessed the curse of dimensionality, the infeasibility of doing auto-differentiation, and the convergence to constants. These findings highlighted some obstacles that are rarely discussed and reminded one that generalizing new architecture to high-order nonlinear PDEs remains a non-trivial task.

On the other hand, we've made intriguing discoveries, opening up potential directions worth exploring. In fact, we've applied various modifications to the PDDLVM model. For instance, we tried implementing periodic activation functions proposed in [8], normalizing inputs, and adding random Fourier features features[9]. We also came across one particularly interesting work called PDE-preserved neural network (PPNN) [6]. Instead of enforcing the physical loss as a soft constraint, it's enforcing physical laws in a more deterministic manner. The author feeds the input into convolution layers with predetermined "differential operator" kernels derived from the finite difference method. Furthermore, we came across several papers like [11] that discuss the non-convergence issue, and [3] tried to address it by designing losses that punish zero gradients (implying a constant solution). Lastly, we reviewed the potential of neural operators; it surprisingly gained some knowledge on the dynamics of the PDE system. Suggesting neural networks' potential to learn complex systems. In fact, we've only implemented a fundamental neural operator, and many improvements have been made. For example, [4] tried to stabilize the long-term prediction, and [13] proposed a mesh-free neural operator. Given the author of PDDLVM also suggested the integration of FNO into the PDDLVM framework, we believe it's one of the more promising directions to explore and hope this work will provide valuable insights for researchers who are interested in this field.

6. References

References

[1] Shaho Abdalla et al. "Thermodynamics, formation dynamics, and structural correlations in the bulk amorphous phase of the phase-field crystal model". In: *The J. Chem. Phys.* 157.16 (Oct. 2022). ISSN: 1089-7690. DOI: 10.1063/5.0114705. URL: http://dx.doi.org/10.1063/5.0114705.

[2] Zheyuan Hu et al. "Tackling the curse of dimensionality with physics-informed neural networks". In: *Neural Networks* 176 (Aug. 2024), p. 106369. ISSN: 0893-6080. DOI: 10.1016/j.neunet.2024.106369. URL: http://dx.doi.org/10.1016/j.neunet.2024.106369.

[3] Raphael Leiteritz and Dirk Pflüger. *How to Avoid Trivial Solutions in Physics-Informed Neural Networks*. 2021. arXiv: 2112.05620 [cs.LG]. URL: https://arxiv.org/abs/2112.05620.

[4] Zhijie Li et al. "Long-term predictions of turbulence by implicit U-Net enhanced Fourier neural operator". In: *Phys. Fluids* 35.7 (July 2023). ISSN: 1089-7666. DOI: 10.1063/5.0158830. URL: http://dx.doi.org/10.1063/5.0158830.

[5] Zongyi Li et al. *Fourier Neural Operator for Parametric Partial Differential Equations*. 2021. arXiv: 2010.08895 [cs.LG]. URL: https://arxiv.org/abs/2010.08895.

[6] Xin-Yang Liu et al. "Multi-resolution partial differential equations preserved learning framework for spatiotemporal dynamics". In: *Commun. Phys.* 7.1 (Jan. 2024). ISSN: 2399-3650. DOI: 10.1038/s42005-024-01521-z. URL: http://dx.doi.org/10.1038/s42005-024-01521-z.

[7] M. Raissi, P. Perdikaris, and G.E. Karniadakis. "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations". In: *J. Comput. Phys.* 378 (2019), pp. 686–707. ISSN: 0021-9991. DOI: https://doi.org/10.1016/j.jcp.2018.10.045. URL: https://www.sciencedirect.com/science/article/pii/S0021999118307125.

[8] Vincent Sitzmann et al. *Implicit Neural Representations with Periodic Activation Functions*. 2020. arXiv: 2006.09661 [cs.CV]. URL: https://arxiv.org/abs/2006.09661.

[9] Matthew Tancik et al. *Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains*. 2020. arXiv: 2006.10739 [cs.CV]. URL: https://arxiv.org/abs/2006.10739.

[10] Arnaud Vadeboncoeur et al. "Fully probabilistic deep models for forward and inverse problems in parametric PDEs". In: *J. Comput. Phys.* 491 (Oct. 2023), p. 112369. ISSN: 0021-9991. DOI: 10.1016/j.jcp.2023.112369. URL: http://dx.doi.org/10.1016/j.jcp.2023.112369.

[11] Sifan Wang, Xinling Yu, and Paris Perdikaris. "When and why PINNs fail to train: A neural tangent kernel perspective". In: *J. Comput. Phys.* 449 (2022), p. 110768. ISSN: 0021-9991. DOI: https://doi.org/10.1016/j.jcp.2021.110768. URL: https://www.sciencedirect.com/science/article/pii/S002199912100663X.

[12] Sifan Wang et al. *An Expert's Guide to Training Physics-informed Neural Networks*. 2023. arXiv: 2308.08468 [cs.LG]. URL: https://arxiv.org/abs/2308.08468.

[13] Zecheng Zhang, Wing Tat Leung, and Hayden Schaeffer. *BelNet: Basis enhanced learning, a mesh-free neural operator*. 2022. arXiv: 2212.07336 [math.NA]. URL: https://arxiv.org/abs/2212.07336.